

谭演锋

✉ hafeng33@gmail.com · ☎ 13427590022 · in <https://www.linkedin.com/in/yanfeng-tan/>

教育背景

美国加州大学圣地亚哥分校 (US News: 21)

计算机科学硕士 GPA: 3.8 / 4.0

2025/09 – 2027/06

华南理工大学

计算机科学学士 GPA: 92.14 / 100

2021/09 – 2025/06

实习经历

Arm

2026/01 – 至今

- 负责 Profiling 工具研发工作, 重构开发 **Causal Profiling, Coz** 工具, 量化关键代码优化对整体系统吞吐量影响。
- 实现 Coz 程序预热后启动分析、收集完整的调用栈信息等方式, 并实现 profile 内核级函数, 提高工具的兼容性和准确性。
- 发现 Coz(Perf) 在 off-CPU 情形下测量不够精准的问题, 引入 Block Sampling 概念, 修改 **Linux 内核 Perf 工具**, 让其能够精准 Profile 多线程的 **on-CPU 执行热点与 off-CPU 等待瓶颈** (如锁竞争、I/O 排队)。
- 利用 Coz 对 llama.cpp 和 MySQL 进行性能剖析。验证团队对 llama.cpp 关于 Arm 平台上性能优化 issue 的准确性。将 MySQL 的瓶颈定位到 CRC 校验函数, 指出 Arm 与 x86 指令差异造成的性能瓶颈。

西门子

2025/04 – 2025/08

- 开发并上线基于 LangChain / LangGraph 的 RAG CT 操作问答系统, 现已在西门子医疗西影力学城正式上线服务。
- 搭建 ETL 流水线, 从对象存储系统抽取文档并转化至 Markdown 文档 (支持嵌套表格等复杂结构), 基于 Milvus 实现支持混合检索的多向量存储与增量嵌入/定向删除, 配合异步任务队列提升约 2 倍嵌入效率。
- 优化结合的稀疏, 稠密向量和 BM25 进行混合检索, 叠加 Reranker, RRF, MRR 算法重排; 引入文档摘要检索和 Query Refine 等策略, 将平均 Correctness 从 76% 优化到 92%, Relevance 与 Groundedness 保持 99%。
- 基于 LangSmith 追踪, 评价 RAG 系统, 按 Correctness、Relevance、Groundedness 对 RAG 系统打分, 完善系统可观测性。

用友网络

2024/07 – 2024/09

- 参与人力资源系统后端模块开发, 服务 **8K+** 员工, 负责跨系统数据迁移与同步能力建设, 保障数据的一致性与完整性。
- 基于 gRPC 和 Python 脚本实现迁移服务, 通过断点续传与增量标记机制, 支持全量/增量迁移, 批处理写入 PostgreSQL。
- 利用 Redis 缓存高频字段, 缓存命中率约 **90%**, 在峰值流量下为 PostgreSQL 减少约 **1K QPS** 负载, p95 延迟下降约 **40%**。
- 通过慢查询, pg_stat_statements 和 EXPLAIN ANALYZE 定位 PostgreSQL 性能瓶颈, 针对热点查询重构索引并改写 SQL, 消除 N+1 查询, 提高 Sql 效率。

项目经历

仿大众点评项目 (GitHub)

- 基于 **SpringBoot, SpringCloud** 等构建核心模块 (商品, 用户, 订单等), 通过 **Nacos** 实现服务注册与配置热加载。并使用 **Sentinel** 对服务进行请求限流, 线程隔离, 服务降级, 保护整体服务可用性。
- 使用 **Redis** 实现登录功能, **Threadlocal** 配合网关校验 token 并刷新, 完成用户认证, 通过拦截器清理 ThreadLocal。
- 基于 **Redis** 和 **Lua** 脚本保障用户 “一人一单”, 解决库存超卖问题; 使用 **乐观锁** 提升并发写入性能。
- 使用 **RabbitMQ+ Lua** 实现异步秒杀下单, 平均耗时由 400ms 降至 100ms。基于 RabbitMQ 死信队列实现订单超时关闭。

Java RPC 框架 (GitHub)

- 基于 TCP 的自定义协议和 **Netty** 实现高性能 RPC 框架。利用 **JDK 动态代理**, 实现无感透明远程调用。
- 基于统一 **SPI** 与工厂模式实现可插拔序列化、负载均衡与容错策略。
- 集成 **Nacos** 作为服务注册中心, 使用线程安全的 **ConcurrentHashMap** 存储本地服务注册信息, 实现服务注册与发现。

高性能矩阵乘法 (GitHub)

- 在 ARM 平台实现优化版 DGEMM 内核, 结合多级 cache blocking、数据打包与基于 **SVE** 的微内核设计, 峰值性能达 **22 GFLOPS**, 较朴素三重循环实现提速超过 **20 倍**, 在大规模矩阵上略优于 **OpenBLAS**。
- 开发基于 CUDA 的 tiled GEMM 内核, 采用共享内存 tiling、二维指令级并行 (2-D ILP) 和如 **32x8** 的线程块形状调优, 在 NVIDIA T4 GPU 上实现约 **3.6–3.8 TFLOPS**, 相对朴素实现提速约 **15 倍**, 性能接近 **cuBLAS** 的约 **80%**。

相关技能

- 编程语言: Java, Python, C/C++, CUDA, SQL (PostgreSQL, MySQL), JavaScript/TypeScript, HTML/CSS, Lua
- 开发框架: Spring Boot, Spring Cloud, Flask, React, Vue, LangChain, LangGraph, FastAPI, Netty, gRPC
- 工具与平台: Git, Docker, Nginx, Linux, Unix, AWS, Redis, MongoDB, RabbitMQ, Elasticsearch, Milvus, Nacos, vLLM, GitLab CI/CD, Ollama